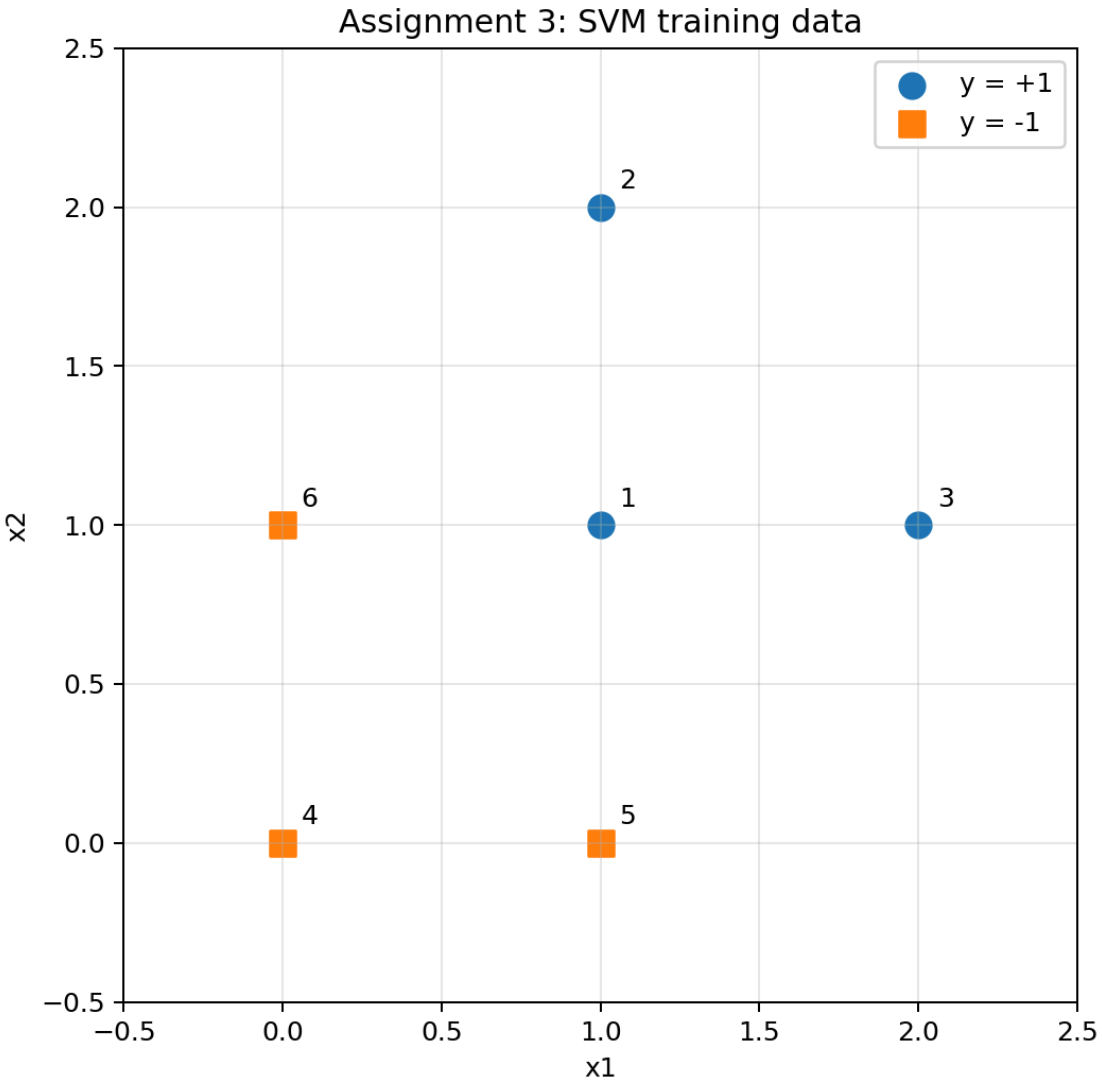
Question 1(a): Support Vectors



The support vectors are points 1, 5, and 6 because they are the closest points to the optimal boundary separating the two classes. These points determine the position of the maximum-margin separating boundary, while the remaining points are farther away.

Question 1(b): Maximum-Margin Separating Boundary and Geometric Margin

For a hard-margin linear SVM, the decision boundary is given by

$$w_1 x_1 + w_2 x_2 + b = 0.$$

The support vectors satisfy

$$y_i(\mathbf{w}^T \mathbf{x}_i + b) = 1.$$

Using the support vectors identified in part (a), we obtain the following equations:

- Point 1: $(1, 1)$, $y = +1$
- Point 5: $(1, 0)$, $y = -1$
- Point 6: $(0, 1)$, $y = -1$

Substituting these points gives

$$\begin{aligned} w_1 + w_2 + b &= 1 \qquad (1) \\ w_1 + b &= -1 \qquad (2) \\ w_2 + b &= -1 \qquad (3) \end{aligned}$$

Subtracting equation (2) from equation (1):

$$\begin{aligned} (w_1 + w_2 + b) - (w_1 + b) &= 1 - (-1) \\ w_2 &= 2. \end{aligned}$$

Similarly, subtracting equation (3) from equation (1):

$$\begin{aligned} (w_1 + w_2 + b) - (w_2 + b) &= 1 - (-1) \\ w_1 &= 2. \end{aligned}$$

Substituting $w_1 = 2$ into equation (2):

$$\begin{aligned} 2 + b &= -1 \\ b &= -3. \end{aligned}$$

Therefore, the weight vector and intercept are

$$\boxed{\mathbf{w} = (2, 2), \quad b = -3.}$$

Substituting these values into the decision boundary equation gives

$$\boxed{2x_1 + 2x_2 - 3 = 0.}$$

The geometric margin is the perpendicular distance from the decision boundary to the closest training points, given by

$$\gamma = \frac{1}{\|\mathbf{w}\|}.$$

Since $\mathbf{w} = (2, 2)$, its Euclidean norm is

$$\begin{aligned} \|\mathbf{w}\| &= \sqrt{w_1^2 + w_2^2} \\ &= \sqrt{2^2 + 2^2} \\ &= \sqrt{8} \\ &= 2\sqrt{2}. \end{aligned}$$

Therefore, the geometric margin is

$$\gamma = \frac{1}{2\sqrt{2}} = \frac{\sqrt{2}}{4} \approx 0.3536.$$

Therefore, the maximum-margin separating boundary is $2x_1 + 2x_2 - 3 = 0$, with a geometric margin of approximately 0.3536, representing the perpendicular distance from the decision boundary to the closest training points.

2(a) K-means Implementation

I implemented the two functions update_assignments() and update_centroids() using NumPy.

update_assignments() assigns each observation to its nearest centroid using squared Euclidean distance. If distances are equal, the centroid with the lowest index is chosen.

update_centroids() updates each centroid using the mean of its assigned observations. If a cluster is empty, its previous centroid is retained.

The implementation is provided in kmeans_template.py.

2(b) K-means Results

K-means was run on the toy dataset of 4,000 observations generated from four groups, using k = 2, 3, 4, 6, and 10.

In [16]:
```
import matplotlib.pyplot as plt
import matplotlib.image as mpimg
from pathlib import Path

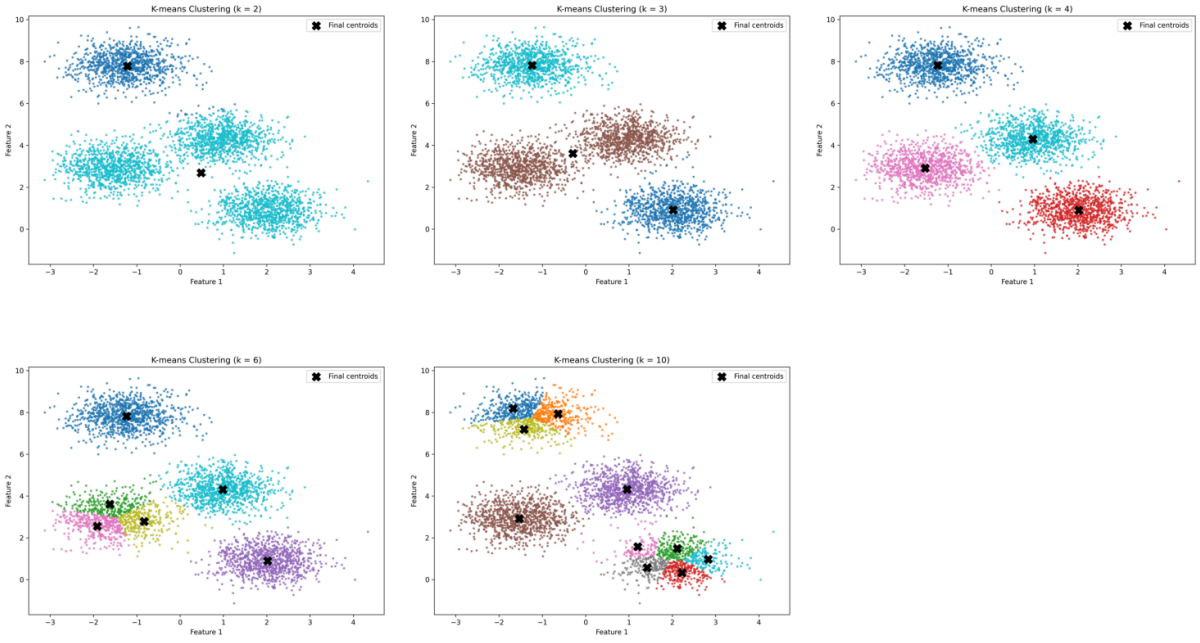
k_values = [2, 3, 4, 6, 10]

fig, axes = plt.subplots(2, 3, figsize=(15, 9))

for ax, k in zip(axes.flat, k_values):
    img = mpimg.imread(f"results/kmeans/kmeans_k{k}.png")
    ax.imshow(img)
    ax.axis("off")

# Hide the unused sixth subplot
axes.flat[-1].axis("off")

plt.tight_layout()
plt.show()
```



As (k) increases, the algorithm divides the data into more clusters. When (k=2) and (k=3), some of the four generating groups are combined into the same cluster. When (k=4), the algorithm successfully identifies all four generating groups. However, when (k=6), the lower-left group is divided into three separate clusters, while the other groups remain intact. Interestingly, when (k=10), that same group is identified as one cluster, while other groups are divided further. This shows that increasing (k) does not necessarily divide the same groups each time, since the results also depend on the initial centroids. Overall, larger values of (k) can lead to unnecessary splitting of the original generating groups.

3(a) Noisy-label SVM Baseline

Independent training and test datasets were generated, each containing 1,000 positive and 1,000 negative observations. The positive observations were generated from a normal distribution with mean ((1,1)) and covariance matrix (I), while the negative observations were generated with mean ((-1,-1)) and covariance matrix (3I).

To introduce noise, positive training labels were flipped with probability 0.35 and negative training labels with probability 0.20. We then trained an RBF SVM with (C=1) and (gamma=0.01). This process was repeated 20 times using the same training and test observations but different corrupted training labels.

The mean accuracy on the clean test set was **82.80%**.

3(b) K-NN Label Repair

To try to improve the SVM's performance, K-NN was used to repair the noisy training labels. For each observation, its five nearest neighbors were found using Euclidean distance,

excluding the observation itself. Then the labels were replaced with the majority label of those neighbors. If distances were equal, observations were selected with the lowest index.

A new SVM was trained using the repaired labels and evaluated on the same clean test set. This was repeated for all 20 trials to compare the baseline SVM with the K-NN repair approach.

In [17]:
```
import pandas as pd

# Load the results
results = pd.read_csv("results/noise_comparison.csv")

# Calculate mean accuracies
baseline_mean = results["baseline_accuracy"].mean()
knn_mean = results["knn_accuracy"].mean()

# Create a summary table
summary = pd.DataFrame({
    "Method": ["Baseline SVM", "SVM with K-NN Repair"],
    "Mean Accuracy": [baseline_mean, knn_mean]
})

display(summary.round(4))

print(f"Difference: {knn_mean - baseline_mean:.4f}")
```

	Method	Mean Accuracy
0	Baseline SVM	0.8280
1	SVM with K-NN Repair	0.8644

```
Difference: 0.0364
```

The baseline SVM had a mean accuracy of 82.80%, while the SVM trained using K-NN repaired labels had a mean accuracy of 86.44%. This is an improvement of 3.64 percentage points, suggesting that K-NN label repair helped improve the SVM's performance. This may be because K-NN uses the majority labels of nearby observations to correct some of the incorrectly labeled training data. However, K-NN may not always improve performance, since observations near the decision boundary could be assigned incorrect labels based on their neighbors.